VORTEX: A LIGHTWEIGHT CROSS-PLATFORM GPU-ACCELERATED ENGINE FOR REAL-TIME OBJECT TRACKING AND RENDERING

A PREPRINT

Prakharkumar Srivastava[*,*] Flam, Bangalore, India IIT Roorkee, India

Nikhil Karthik[*] Flam, Bangalore, India IIIT Dharwad, India

Aman Agrawal[*] Flam, Bangalore, India

Aloori Raj Aryan Flam, Bangalore, India IIT BHU, Varanasi, India

April 15, 2026

ABSTRACT

Real-time 6-DoF pose estimation on mobile and embedded platforms is constrained by limited computational resources and the need for geometric robustness. While dense SLAM systems and learned pose regression models provide high accuracy, they are often impractical for edge deployment. Classical feature-based tracking offers a lightweight alternative but relies on algebraic formulations, such as homography estimation, that lack explicit 3D constraints and degrade under viewpoint changes, scale variation, and partial visibility.

We present ***VORTEX*** (Visual Object Rendering and Tracking Engine for Cross-Platform), a hardware- and geometry-aware tracking framework designed for edge devices. The system combines a SIMD-optimized CPU pipeline with a GPU-accelerated feature pipeline for efficient keypoint detection, descriptor extraction, and matching. Building on this, we introduce a render-and-track formulation on the $\mathrm{SE}(3)$ manifold, enabling direct 2D–3D correspondence lifting through synthetic reference rendering.

To improve stability, we propose a physics-inspired pose refinement method in which correspondences are modeled as virtual springs. Residual magnitudes define adaptive weights, yielding a weighted Gauss–Newton update that suppresses outliers and improves convergence under partial visibility and non-uniform feature distributions.

Experiments on Android devices demonstrate that while GPU acceleration improves throughput, the proposed geometry-aware refinement significantly enhances pose stability and robustness for real-time 6-DoF tracking without persistent mapping.

Keywords 6-DoF pose estimation · Edge computing · Hardware-aware vision · GPU-accelerated feature tracking · Render-and-track · SE(3) pose optimization · Mobile augmented reality

1 Introduction

Accurate six degrees-of-freedom (6-DoF) pose estimation is fundamental to augmented reality (AR), where virtual content must remain geometrically aligned with physical objects under strict real-time constraints. Although modern visual SLAM systems and learned pose regression models achieve high accuracy [1, 2, 3, 4], they typically rely on dense mapping, global optimization, or large-scale training. Such approaches introduce significant computational and memory overhead, making them difficult to deploy on mobile and embedded platforms where latency, power, and resource constraints are critical.

Classical feature-based tracking pipelines provide a computationally efficient alternative and remain widely used in mobile vision systems [5, 6]. These pipelines typically consist of image preprocessing, keypoint detection, descriptor

[*]Equal contribution.

extraction, and feature matching, followed by motion estimation using algebraic formulations such as homography estimation or optical flow. While lightweight, two major challenges remain. First, feature extraction and matching often dominate the runtime on resource-constrained devices, creating a critical computational bottleneck. Second, purely algebraic formulations lack explicit 3D geometric reasoning, making them susceptible to instability under out-of-plane rotations, rapid scale changes, and planar degeneracies.

In this work, we present ***VORTEX*** (Visual Object Rendering and Tracking Engine for Cross-Platform), a framework that investigates how hardware-aware acceleration and geometric reasoning can be jointly leveraged to enable efficient and stable tracking on edge devices. We develop a three-stage tracking framework that progressively addresses both computational and geometric limitations. First, we implement a highly optimized CPU-based algebraic tracking pipeline that exploits SIMD vectorization (e.g., ARM NEON) to accelerate feature detection, descriptor computation, and matching. Second, to further alleviate feature processing bottlenecks, we introduce a GPU-accelerated pipeline in which grayscale conversion, keypoint detection, and descriptor extraction are executed using custom fragment shaders, while descriptor matching is performed in parallel using compute shaders.

While these optimizations significantly improve feature processing throughput, algebraic tracking methods remain fundamentally limited by the instability of planar homography estimation. To address this limitation, we introduce a geometry-aware render-and-track refinement stage formulated on the $\mathrm{SE}(3)$ pose manifold [7, 8]. Our approach renders synthetic reference views of known object geometry at discrete depth hypotheses and stores both appearance information and object-space coordinates. This representation enables direct lifting of image correspondences to 2D–3D constraints.

Building on this formulation, we propose a physics-inspired pose refinement method in which 2D–3D correspondences are modeled as a system of virtual springs in image space. Each correspondence induces a residual whose magnitude defines a spring length, while the smallest residual acts as a reference equilibrium. The resulting system generates tension forces that adaptively reweight correspondences based on their relative consistency. By linearizing these forces on the $\mathrm{SE}(3)$ manifold, the method yields a weighted Gauss–Newton update, enabling robust pose refinement that naturally suppresses outliers without requiring hand-crafted robust loss functions.

To efficiently combine these components, the system employs a hybrid detect–refine–track state machine that integrates global feature matching with lightweight Lucas–Kanade optical flow for frame-to-frame updates. This design enables robust relocalization while maintaining real-time performance on mobile-class hardware without relying on learned models or persistent mapping.

Contributions. The main contributions of this work are:

- A hardware-aware feature tracking foundation optimized for mobile devices using SIMD-accelerated CPU processing.
- A fully GPU-resident feature processing pipeline that performs keypoint detection and descriptor construction using fragment shaders and parallel descriptor matching using compute shaders.
- A geometry-aware render-and-track formulation that enables explicit 2D–3D correspondence lifting through synthetic reference rendering.
- A physics-inspired, tension-based pose refinement method formulated on $\mathrm{SE}(3)$, which yields a weighted Gauss–Newton solver with adaptive correspondence weighting.
- A hybrid tracking framework that combines detection, refinement, and optical flow for real-time, robust 6-DoF tracking without persistent mapping or learned priors.

2 Related Work

Visual SLAM and Dense Pose Estimation. Visual SLAM systems estimate camera motion while simultaneously reconstructing scene geometry through feature-based or dense optimization pipelines [1, 2, 4, 3]. These approaches typically rely on persistent mapping, global bundle adjustment, and long-term state estimation. While highly accurate, such pipelines incur substantial computational and memory overhead, particularly for dense direct methods that operate on per-pixel photometric alignment. As a result, deploying full SLAM systems on mobile or embedded devices often introduces significant latency and power consumption, making them unsuitable for real-time augmented reality scenarios where only object-level pose tracking is required.

Model-Based Tracking for Textureless Objects. Model-based approaches leverage known object geometry to estimate pose by aligning projected model features with image observations. A representative example is the edge-based

tracking framework proposed by Choi and Christensen, which aligns projected contours with image gradients using a particle filtering formulation [9]. Such methods are effective for textureless objects and exploit strong geometric priors. However, they typically rely on dense edge measurements and iterative evaluation of multiple pose hypotheses, resulting in high computational cost. More broadly, edge- and contour-based tracking methods remain sensitive to noise, clutter, and partial occlusion, limiting their robustness in real-world mobile scenarios [**?**].

Feature-Based Object Tracking. Classical feature-based pipelines estimate motion by detecting keypoints, computing local descriptors, and matching them across frames. Efficient binary descriptors such as ORB [10] and BRISK [11] enable fast matching using Hamming distance and are widely used in real-time systems [12, 13]. Pose is typically recovered using algebraic formulations such as planar homography estimation or Perspective-n-Point (PnP) solvers [14, 15, 16]. While computationally efficient, these approaches rely purely on image-space correspondences and lack explicit geometric reasoning. Consequently, they often exhibit instability under large viewpoint changes, rapid scale variations, and planar degeneracies.

Learning-Based Pose Estimation. Recent work has explored deep neural networks that regress object pose directly from RGB images. These methods can achieve robustness to appearance variation, occlusion, and illumination changes when trained on large annotated datasets. However, they require substantial training data and computational resources, and inference often depends on GPU acceleration. Furthermore, adapting such models to new objects typically requires retraining or fine-tuning, limiting their flexibility in deployment scenarios with constrained compute budgets.

Render-and-Compare Pose Tracking. Render-and-compare approaches estimate pose by rendering synthetic views of known object geometry and minimizing discrepancies with the observed image. By incorporating explicit geometric reasoning, these methods can achieve stable pose estimation through photometric or geometric alignment. However, dense render-and-optimize formulations typically involve iterative optimization over high-dimensional image representations, making them computationally expensive and difficult to deploy efficiently on mobile hardware.

Optimization-Based Pose Refinement. Pose refinement is commonly formulated as a nonlinear least-squares problem on the $\mathrm{SE}(3)$ manifold and solved using Gauss–Newton or Levenberg–Marquardt optimization [17, 18]. While effective, these methods treat all correspondences uniformly or rely on predefined robust loss functions (e.g., Huber or Cauchy) to mitigate outliers [19, 20]. Such approaches do not explicitly exploit the relative structure of residuals within a given frame, and often require careful tuning of loss parameters to balance robustness and convergence.

Hybrid Tracking Frameworks. To balance robustness and efficiency, hybrid systems combine global feature matching with lightweight local tracking methods such as Lucas–Kanade optical flow. These approaches enable reliable relocalization while maintaining efficient frame-to-frame tracking. Building on this principle, our work explores a hierarchy of tracking configurations tailored for edge devices: (i) a SIMD-optimized CPU pipeline for efficient feature processing, (ii) a GPU-accelerated pipeline using programmable shaders for feature extraction and parallel matching, and (iii) a geometry-aware $\mathrm{SE}(3)$ render-and-track framework that integrates synthetic reference rendering with adaptive, correspondence-driven pose refinement.

3 System Overview

The proposed framework enables efficient and robust 6-DoF tracking on mobile and embedded devices by combining hardware-aware feature processing with geometry-aware pose refinement. The system is structured as a progressive hierarchy of three tracking configurations: (i) a CPU-based algebraic pipeline, (ii) a GPU-accelerated feature processing pipeline, and (iii) a geometry-aware render-and-track framework operating on the $\mathrm{SE}(3)$ pose manifold. Figure 1 illustrates the overall system architecture.

All configurations share a common feature-based tracking backbone. Incoming frames are converted to grayscale and processed using a FAST-based keypoint detector, followed by non-maximum suppression. Local binary descriptors are computed for the detected features and matched across frames using efficient Hamming-distance comparisons. The resulting correspondences are used to estimate motion via an algebraic alignment stage based on homography estimation, while pyramidal Lucas–Kanade optical flow provides efficient frame-to-frame tracking of inlier points.

The first configuration executes the entire pipeline on the CPU and leverages SIMD vectorization to accelerate feature extraction, descriptor computation, and matching. The second configuration introduces GPU acceleration by offloading keypoint detection and descriptor construction to programmable shaders, while descriptor matching is performed in parallel using compute shaders. Both configurations retain the same algebraic motion model and primarily differ in their computational efficiency.

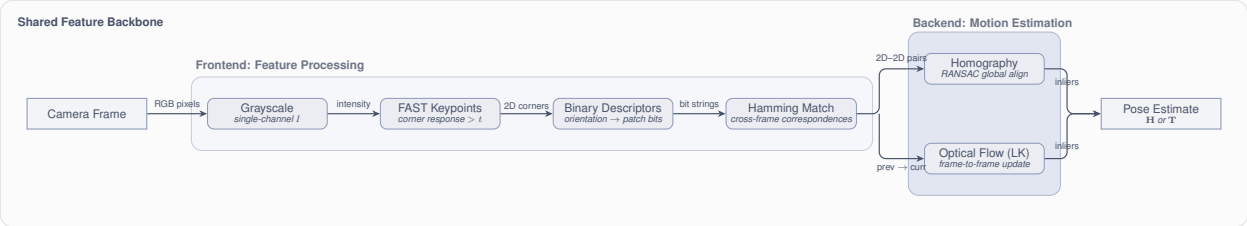

Figure 1: Shared feature extraction backbone common to all three system configurations. Incoming frames are converted to grayscale, processed through FAST-based keypoint detection, binary descriptor computation, and Hamming-distance matching. The architecture is cleanly divided into a frontend feature processing pipeline and a backend motion estimation solver, where correspondences feed either a homography solver (global alignment) or a Lucas–Kanade tracker (frame-to-frame update) to yield the final pose estimate.

The third configuration augments this pipeline with a geometry-aware pose refinement stage. Synthetic reference views of the known object are rendered at multiple pose hypotheses, enabling direct lifting of image correspondences to 2D–3D constraints. These correspondences are used to refine the pose on $\mathrm{SE}(3)$ through an iterative optimization procedure, incorporating adaptive correspondence weighting to improve robustness under noise, outliers, and viewpoint changes.

Together, these configurations provide a unified framework for analyzing the trade-offs between purely algebraic tracking, hardware-accelerated feature processing, and geometry-aware pose refinement on resource-constrained devices.

4 Algebraic Tracking Pipeline

The CPU and GPU tracking configurations share a common algebraic formulation based on feature correspondences and planar homography estimation. This formulation enables efficient frame-to-frame motion estimation, but lacks explicit geometric constraints, which can lead to instability under challenging viewing conditions such as partial visibility or large perspective changes.

4.1 Feature Detection and Matching

Given an input frame, salient keypoints are detected using the FAST corner detector [12]. For each keypoint, a binary descriptor is computed to encode local image structure. In our implementation, BRISK descriptors [11] are employed due to their robustness and compact binary representation.

Correspondences between frames are established by comparing descriptors using Hamming distance. Let d_i and d_j denote two binary descriptors. Their similarity is computed as:

$$D(d_i, d_j) = \mathrm{popcount}(d_i \oplus d_j),$$

where $\oplus$ denotes the bitwise XOR operation and $\mathrm{popcount}(\cdot)$ counts the number of set bits. Correspondences are selected by identifying nearest neighbors under this metric, optionally filtered using ratio tests to reject ambiguous matches.

This operation is efficiently implemented on the CPU using SIMD instructions and on the GPU using parallel compute kernels, while preserving identical matching semantics across both execution paths.

4.2 Homography-Based Motion Estimation

Given a set of matched feature points $(x_i, y_i) \leftrightarrow (x'_i, y'_i)$, motion is estimated using a planar homography transformation:

$$\lambda \begin{bmatrix} x'_i \\ y'_i \\ 1 \end{bmatrix} = H \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix},$$

where $H \in \mathbb{R}^{3\times 3}$ is the homography matrix.

Stacking constraints from all correspondences yields the homogeneous system:

$$A\mathbf{h} = 0,$$

where $\mathbf{h}$ is the vectorized form of H. The solution is obtained via singular value decomposition (SVD) as the right singular vector corresponding to the smallest singular value.

To improve robustness, homography estimation is performed within a RANSAC framework, which iteratively samples minimal subsets of correspondences and selects the model with the largest consensus set. After initialization, frame-to-frame tracking is maintained using pyramidal Lucas–Kanade optical flow [21], reducing the need for repeated feature detection.

4.3 Numerical Instability Under Partial Visibility

Although computationally efficient, homography estimation becomes ill-conditioned when feature correspondences are spatially localized. In such cases, the matrix A exhibits strong column correlations, leading to a poorly conditioned system.

The sensitivity of the solution can be quantified using the condition number $\kappa(A)$:

$$\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}},$$

where $\sigma_{\max}$ and $\sigma_{\min}$ are the largest and smallest singular values of A, respectively. A large condition number implies that small perturbations in the input correspondences can produce large variations in the estimated homography:

$$\|\Delta\mathbf{h}\| \leq \kappa(A)\|\Delta\mathbf{b}\|.$$

This situation commonly arises when:

- Only a small portion of the object is visible
- Feature points are clustered within a limited region
- The scene exhibits near-planar degeneracy

Under these conditions, the system lacks sufficient spatial diversity to constrain all degrees of freedom of the homography. Consequently, even small tracking errors introduced by Lucas–Kanade optical flow can result in large distortions in the estimated transformation.

Additionally, RANSAC becomes less reliable when the number of correspondences is low. The probability of sampling a valid minimal subset decreases, and the consensus set becomes unstable. This leads to intermittent tracking failures and large pose fluctuations.

4.4 Limitations of Algebraic Formulation

The core limitation of homography-based tracking lies in its purely algebraic nature. The formulation does not explicitly enforce 3D geometric consistency and treats correspondences only as 2D projective constraints. As a result:

- Depth and scale are implicitly coupled and poorly constrained
- Out-of-plane rotations are not accurately modeled
- Degenerate configurations lead to unstable solutions

These limitations motivate the transition to a geometry-aware formulation, where correspondences are lifted to 2D–3D constraints and pose is optimized directly on the $\mathrm{SE}(3)$ manifold. The following section introduces this formulation and demonstrates how it improves robustness through explicit geometric reasoning and adaptive optimization.

5 Hardware-Aware Implementations

To achieve real-time performance on mobile and edge devices, the proposed system incorporates hardware-aware optimizations tailored for both CPU and GPU execution. While the underlying algebraic formulation remains consistent across implementations, computational stages are reorganized to exploit the architectural strengths—and mitigate the bottlenecks—of each processing unit.

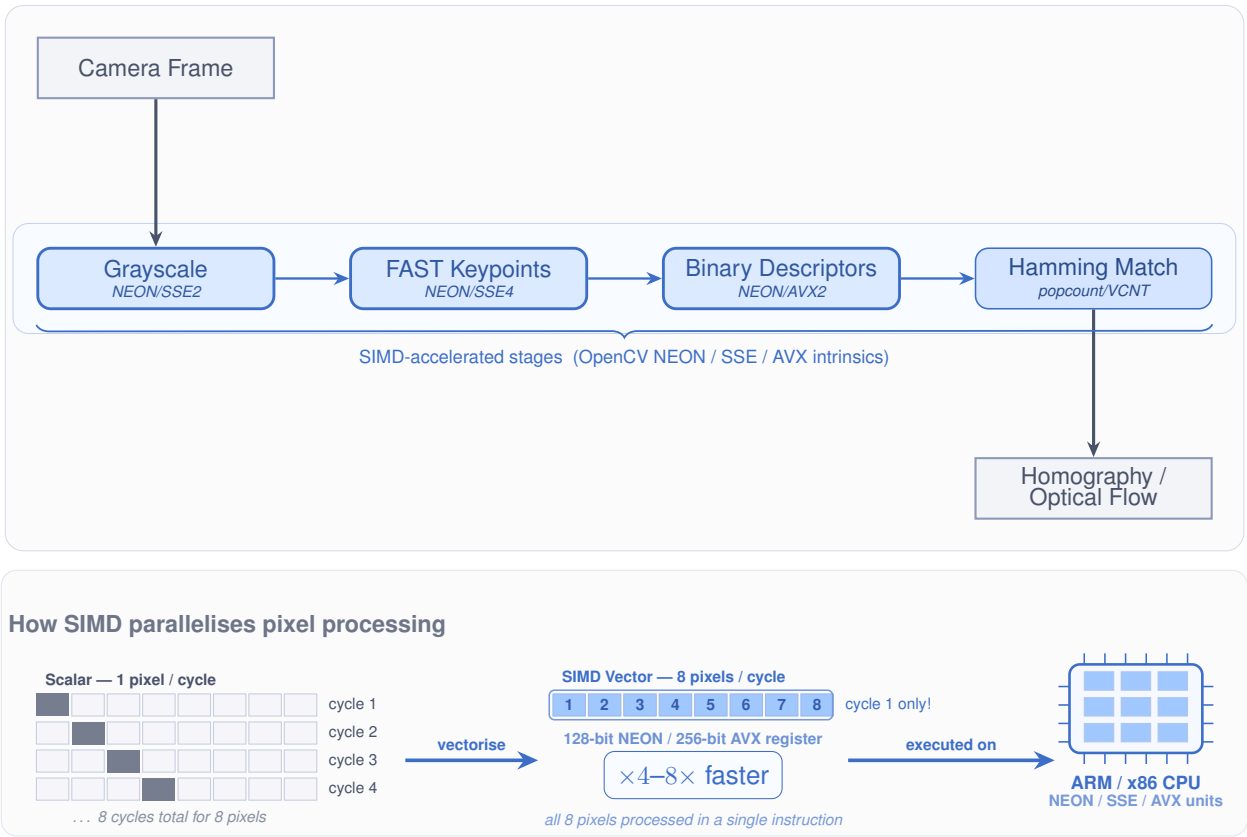

Figure 2: CPU/SIMD pipeline configuration. Grayscale conversion, FAST keypoint detection, binary descriptor computation, and Hamming-distance matching are vectorised via SIMD intrinsics (ARM NEON on mobile, SSE4/AVX2 on desktop). The lower panel contrasts scalar execution—one pixel per cycle across 8 cycles—with SIMD vector execution where a single 128-bit NEON or 256-bit AVX2 instruction processes all 8 pixels simultaneously, yielding 4–8× throughput on these stages.

5.1 CPU Implementation

The CPU-based pipeline is designed to minimize memory latency and maximize instruction-level parallelism. Image frames are acquired directly from the native camera interface, enabling operation on CPU-resident buffers and avoiding costly data transfers. This zero-copy design eliminates synchronization overhead associated with GPU offloading.

As illustrated in Figure 2, the most computationally intensive stages—grayscale conversion, FAST keypoint detection, binary descriptor construction, and Hamming-distance matching—are accelerated using Single Instruction, Multiple Data (SIMD) vectorization. By leveraging 128-bit ARM NEON intrinsics on mobile processors (or 256-bit AVX2 on desktop architectures), the pipeline transitions from scalar processing to vectorized execution, enabling multiple pixels to be processed per cycle.

Binary descriptor matching is particularly well-suited to SIMD execution. Descriptor comparisons are implemented using vectorized bitwise XOR operations followed by hardware population-count instructions (e.g., `VCNT` or `popcount`). Combined with cache-aligned memory layouts and compiler-assisted auto-vectorization, this approach yields a $4\times$–$8\times$ improvement in feature processing throughput.

Once correspondences are established, motion estimation—via RANSAC-based homography estimation and pyramidal Lucas–Kanade optical flow—is executed on the CPU. These stages involve iterative and branch-heavy computations, which benefit from the CPU's low-latency cache hierarchy and efficient branch prediction.

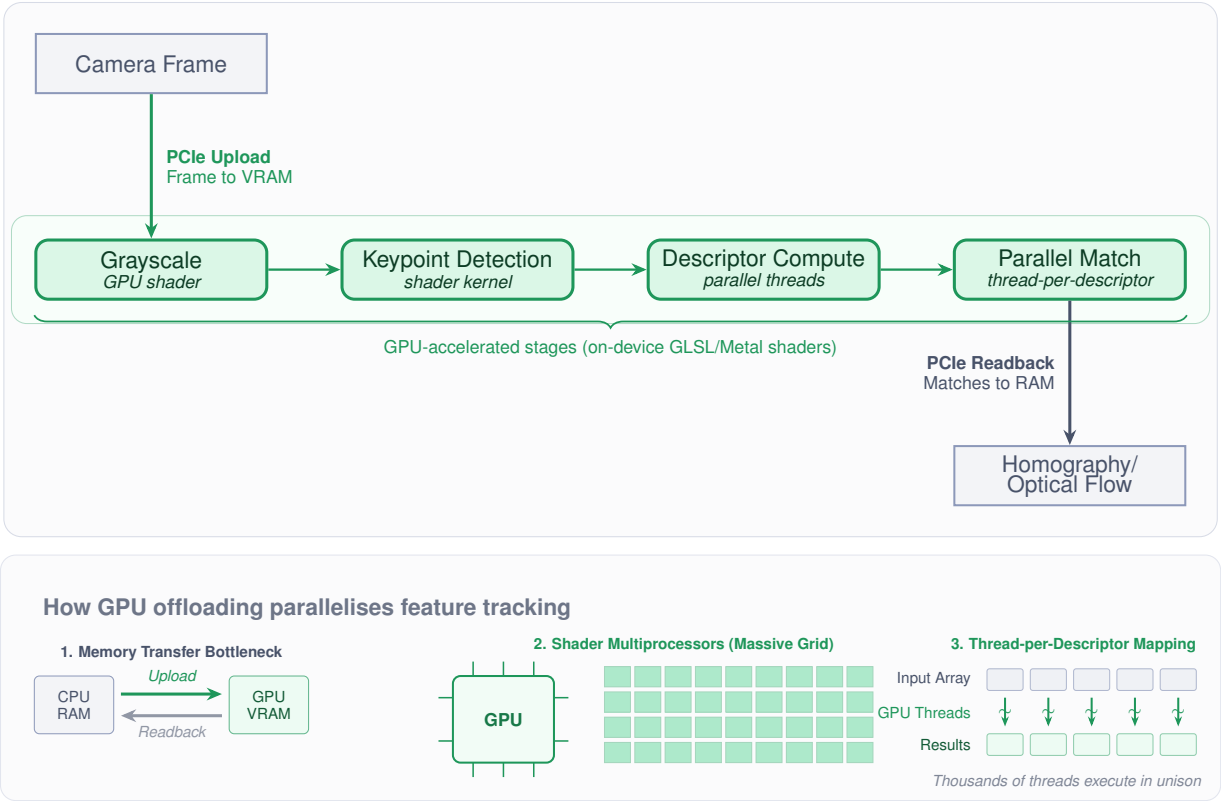

Figure 3: GPU-accelerated configuration. The pipeline is restructured so that Grayscale conversion, Keypoint detection, descriptor computation, and Hamming-distance matching are all offloaded to programmable GPU shaders. The camera frame is uploaded once to VRAM. Thousands of threads process the image in parallel, and only the compact match results are read back to the CPU for homography and optical flow.

5.2 GPU Implementation

The GPU implementation offloads feature extraction and descriptor matching to programmable shader pipelines, trading low-latency sequential execution for high-throughput parallel processing.

As shown in Figure 3, a key challenge in GPU acceleration is minimizing memory transfer overhead. To address this, the camera frame is uploaded to GPU memory once per frame, after which all intermediate processing is performed entirely on the GPU using a sequence of GLSL or Metal shaders with ping-pong buffering. This design ensures that high-bandwidth intermediate data remains in GPU memory.

The GPU pipeline parallelizes the following stages:

- **Grayscale Conversion and Smoothing:** Implemented as a fragment shader pass.
- **Keypoint Detection and NMS:** Parallel evaluation of local intensity contrasts for FAST corner detection.
- **Descriptor Extraction:** Efficient texture sampling to construct binary descriptors.
- **Parallel Matching:** A compute shader employing a thread-per-descriptor model, where thousands of threads simultaneously compute Hamming distances across descriptor sets.

While GPUs excel at data-parallel operations, iterative algorithms such as RANSAC and pyramidal Lucas–Kanade tracking suffer from control-flow divergence and synchronization overhead. Consequently, the system adopts a hybrid execution strategy: the GPU reduces dense image data into a compact set of correspondences, which are then transferred back to the CPU. The CPU subsequently performs motion estimation using its efficient control-flow handling.

Together, these hardware-aware implementations enable the system to adapt to the available compute resources, balancing SIMD-optimized CPU execution with massively parallel GPU processing. This hybrid design ensures efficient feature processing while maintaining real-time performance across a range of mobile and embedded platforms.

6 Geometry-Aware SE(3) Tracking

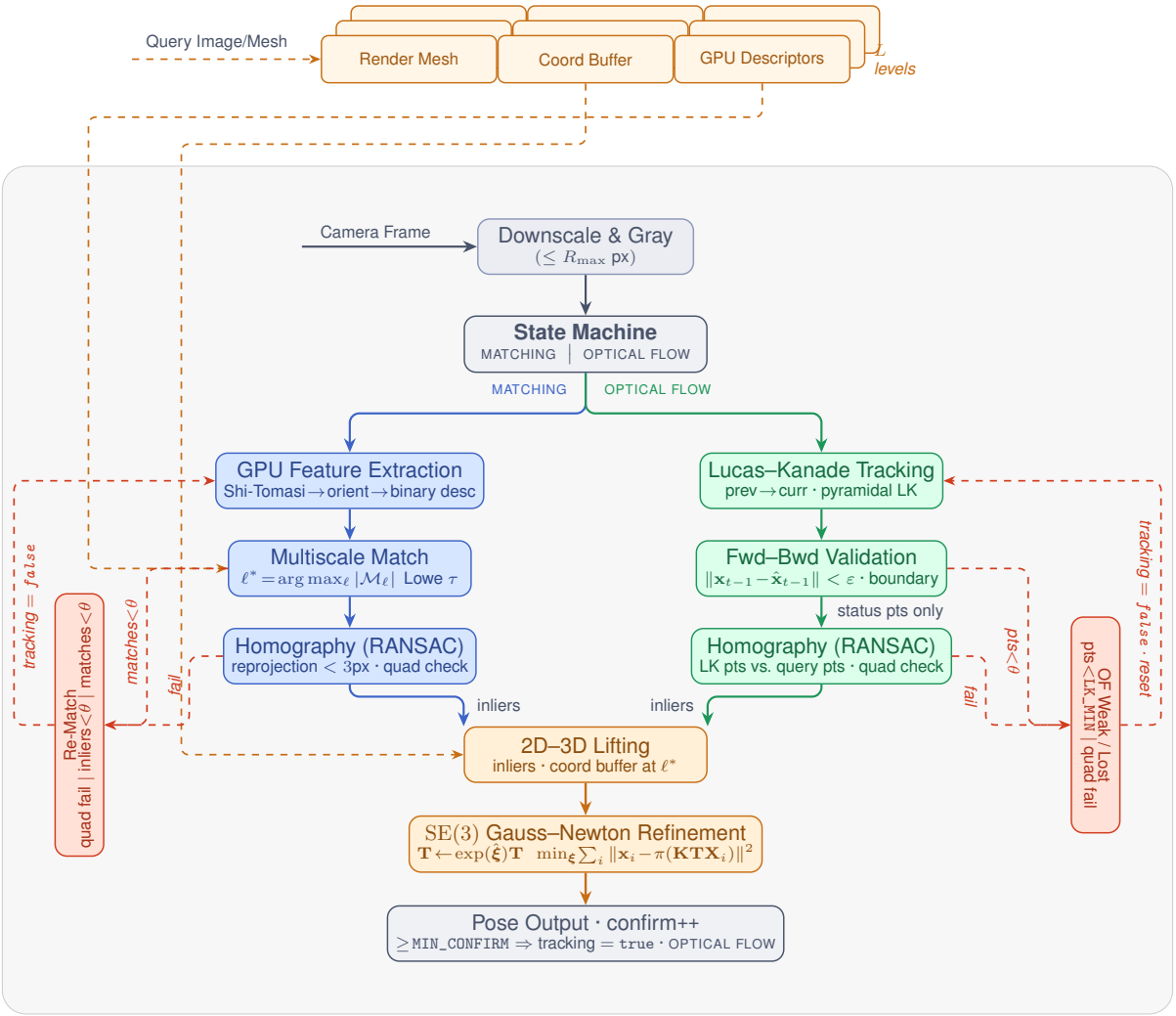

Figure 4: SE(3) render-and-track pipeline. **Top:** A stack of L rendered reference meshes with object-space coordinate buffers and GPU descriptors. **Centre:** The camera frame is downscaled and converted to grayscale, then routed by the state machine to either the global matching branch (left, blue) or the Lucas–Kanade optical-flow branch (right, green). **Merge:** Inliers from both branches are lifted to 3D via the coordinate buffer and refined through SE(3) Gauss–Newton optimisation. **Failure handling:** Dashed red paths loop failed matches or lost tracks back to the corresponding entry points.

While algebraic tracking based on homography estimation provides efficient frame-to-frame motion estimation, it lacks explicit geometric constraints and becomes unstable under large viewpoint changes, scale variation, or partial occlusions. To address these limitations, we introduce a geometry-aware render-and-track formulation that explicitly estimates pose on the SE(3) manifold. As illustrated in Figure 4, the system integrates synthetic rendering with a state machine that dynamically switches between global matching and local optical flow.

6.1 Multiscale Synthetic Reference Rendering

Instead of relying solely on 2D templates, the proposed method renders a set of L synthetic reference views of the object at multiple depth (scale) hypotheses. For each level, the system generates:

- A textured appearance buffer
- A descriptor map for feature matching

- A dense coordinate buffer storing object-space points

Let $\mathbf{X} \in \mathbb{R}^3$ denote a point on the object surface. During rendering, this point is projected into image space using:

$$\mathbf{x} \sim \mathbf{K}[\mathbf{R} \mid \mathbf{t}]\mathbf{X}$$

where $\mathbf{K}$ is the intrinsic matrix and $(\mathbf{R}, \mathbf{t}) \in \mathrm{SE}(3)$.

The coordinate buffer encodes $\mathbf{X}$ per pixel, enabling a direct mapping between image coordinates and 3D object geometry.

6.2 Dual-Branch State Machine

To ensure efficiency, incoming frames are processed through a two-branch state machine:

- **Global Matching Branch:** Activated during initialization or recovery. Features are extracted and matched against the multiscale reference stack. The optimal scale is selected as:

$$\ell^* = \arg\max_{\ell} |\mathcal{M}_\ell|$$

  where $\mathcal{M}_\ell$ denotes valid matches at level ℓ.

- **Optical Flow Branch:** During stable tracking, correspondences are propagated using pyramidal Lucas–Kanade (LK) optical flow [21]. Forward-backward consistency is enforced:

$$\|\mathbf{x}_{t-1} - \hat{\mathbf{x}}_{t-1}\| < \varepsilon$$

In both branches, correspondences are filtered using RANSAC-based homography validation to retain only geometrically consistent inliers.

6.3 2D–3D Correspondence Lifting

Each validated feature correspondence is lifted to a 2D–3D pair:

$$(\mathbf{x}_i, \mathbf{X}_i)$$

by querying the coordinate buffer. This eliminates the need for triangulation or back-projection and directly provides constraints for pose estimation.

6.4 Tension-Based Pose Refinement on $\mathrm{SE}(3)$

Given correspondences $(\mathbf{x}_i, \mathbf{X}_i)$, we define the reprojection function:

$$\mathbf{p}_i(\mathbf{T}) = \pi(\mathbf{K}\mathbf{T}\mathbf{X}_i)$$

and residual:

$$\mathbf{e}_i = \mathbf{x}_i - \mathbf{p}_i(\mathbf{T}), \quad d_i = \|\mathbf{e}_i\|$$

Spring Model Formulation Each correspondence is modeled as a virtual spring in image space:

- Current length: d_i
- Natural length:

$$L = \min_i d_i$$

The energy of each spring is defined as:

$$E_i = \frac{\lambda}{2}(d_i - L)^2$$

Gradient Derivation Applying the chain rule:

$$\frac{dE_i}{d\boldsymbol{\xi}} = \frac{dE_i}{dd_i} \cdot \frac{dd_i}{d\mathbf{e}_i} \cdot \frac{d\mathbf{e}_i}{d\boldsymbol{\xi}}$$

we obtain:

$$\frac{dE_i}{dd_i} = \lambda(d_i - L), \quad \frac{dd_i}{d\mathbf{e}_i} = \frac{\mathbf{e}_i^T}{d_i}, \quad \frac{d\mathbf{e}_i}{d\boldsymbol{\xi}} = -\mathbf{J}_i$$

Combining:

$$\nabla E = \sum_i \mathbf{J}_i^T \left(\lambda \frac{d_i - L}{d_i} \mathbf{e}_i \right)$$

Weighted Gauss–Newton Formulation Defining the weight:

$$w_i = \frac{d_i - L}{d_i}$$

the optimization becomes:

$$E(\mathbf{T}) = \sum_i w_i \|\mathbf{e}_i\|^2$$

Linearizing:

$$\mathbf{e}_i(\boldsymbol{\xi} + \delta) \approx \mathbf{e}_i + \mathbf{J}_i \delta$$

leads to the normal equations:

$$(\mathbf{J}^T \mathbf{W} \mathbf{J})\delta = \mathbf{J}^T \mathbf{W} \mathbf{e}$$

where $\mathbf{W} = \mathrm{diag}(w_i)$.

Pose Update The pose is updated using the exponential map:

$$\mathbf{T} \leftarrow \exp(\delta^\wedge)\mathbf{T}$$

Interpretation This formulation can be interpreted as a physics-inspired weighted least-squares system:

- Small residuals ($d_i \approx L$): low influence
- Large residuals: strong corrective force

Unlike standard Gauss–Newton or fixed robust losses, the weighting is adaptive and derived directly from the residual structure of the current frame.

6.5 Failure Handling and System Integration

The system is designed to be fault-tolerant. If RANSAC produces insufficient inliers or the number of tracked LK points falls below a threshold (`LK_MIN`), tracking is reset and the system re-enters the global matching branch.

Successful pose estimates increment a confirmation counter, and once a minimum threshold ($\geq$ `MIN_CONFIRM`) is reached, the system resumes efficient optical flow tracking.

Implementation Details The rendering stage is implemented using a lightweight cross-platform abstraction built on the *Sokol* framework. On mobile devices, OpenGL ES 3 is used for efficient GPU execution. This ensures that coordinate buffer generation remains portable while fully exploiting available GPU hardware.

7 Experimental Setup

To evaluate the proposed tracking framework, we conducted experiments on Android-based mobile devices using a native application pipeline. The evaluation considers three system configurations: (i) a CPU-based algebraic tracking pipeline, (ii) a GPU-accelerated feature processing pipeline, and (iii) the proposed geometry-aware $\mathrm{SE}(3)$ render-and-track pipeline with tension-based pose refinement.

All experiments were performed using real-time camera input under varying conditions of viewpoint, scale, and partial visibility. The objective is to analyze both computational performance and tracking robustness under realistic constraints encountered on mobile and edge devices.

7.1 Evaluation Metrics

The evaluation focuses on the following metrics:

- **Runtime performance:** Average processing time per frame and achieved frames per second (FPS).
- **Tracking stability:** Visual consistency of the estimated pose under viewpoint changes and partial visibility.
- **Tracking accuracy:** Temporal consistency of the estimated pose across consecutive frames.
- **System footprint:** Computational load and memory transfer overhead between CPU and GPU.

7.2 Hardware Profiling: CPU vs. GPU

Table 1 and Figure 5 summarize the performance characteristics of the SIMD-accelerated CPU pipeline and the GPU compute shader pipeline. GPU timings are obtained from on-device measurements collected across 345 frames (mean keypoint count: ≈360).

Table 1: Average computational time per frame for individual pipeline stages across mobile hardware. GPU timings are measured on-device (345 frames, Android). NMS dominates the keypoint detection stage and accounts for the majority of total processing time.

Pipeline Stage	CPU (SIMD)	GPU (Measured)	Speedup
Memory Transfer (Upload)	—	0.14 ms	—
Grayscale Conversion	1.8 ms	1.40 ms	1.3×
FAST Keypoint Detection	4.2 ms	1.38 ms	3.0×
Non-Maximum Suppression	—	12.41 ms	—
Orientation Assignment	—	2.62 ms	—
Binary Descriptors	4.8 ms	4.16 ms	1.2×
Hamming Matching	3.0 ms	—	—
Total (measured)	**13.8 ms**	**22.11 ms**	—

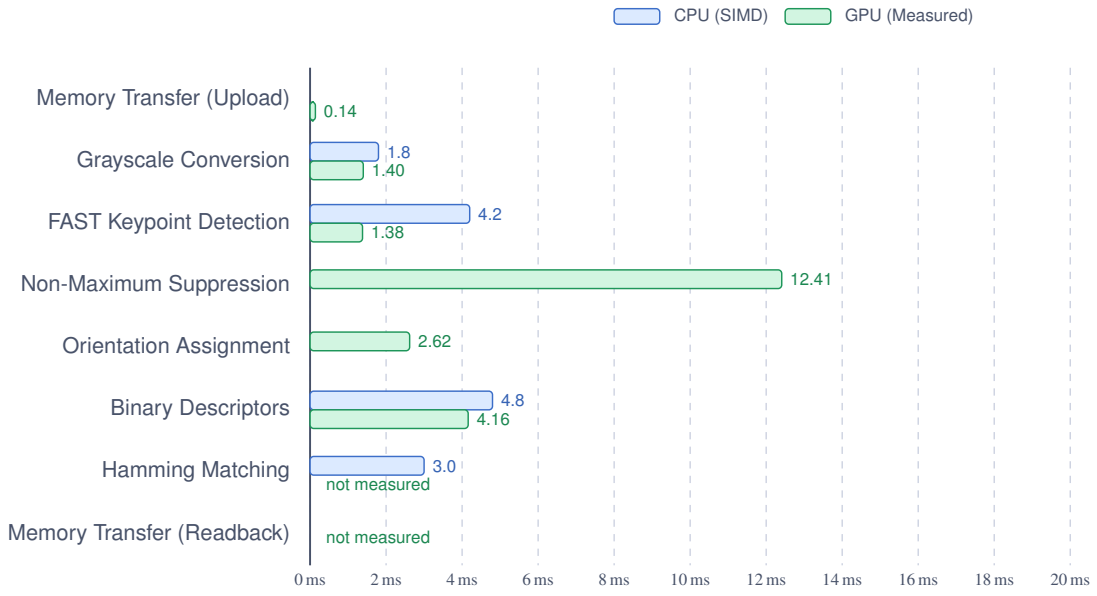

Figure 5: Per-stage execution times comparing the CPU (SIMD) baseline against on-device GPU measurements (345 frames). The axis spans 0–20 ms to accommodate the dominant NMS stage (12.41 ms avg.), which scales with keypoint count (≈360 mean). Stages absent from one pipeline are labelled accordingly.

As shown, the GPU implementation significantly accelerates dense, data-parallel stages such as keypoint detection and descriptor computation. However, achieving high overall throughput requires careful management of memory transfers, particularly the 1.7 ms round-trip latency associated with GPU–CPU communication.

To further characterize performance under real deployment conditions, Table 2 reports per-stage timings measured directly on the Android application across 345 frames. The dataset is divided into two operational regimes: a *detection phase*, where the system searches for the target (93 frames, mean ≈227 keypoints), and a *tracked phase*, where pose estimation is stable (252 frames, mean ≈409 keypoints).

Table 2: On-device measured per-stage timing (mean, ms) for the Android feature extraction pipeline across 345 profiled frames. The detection phase operates on a lower keypoint count (≈227) than the tracked phase (≈409), explaining the approximately $1.9\times$ increase in NMS and descriptor cost during active tracking.

Pipeline Stage	**Detection Phase**	**Tracked Phase**	**Ratio**
Frame Copy	0.12 ms	0.14 ms	$1.2\times$
Blur / Grayscale	1.08 ms	1.52 ms	$1.4\times$
Keypoint Detection	1.07 ms	1.50 ms	$1.4\times$
NMS	7.58 ms	14.19 ms	$1.9\times$
Orientation Assignment	1.83 ms	2.92 ms	$1.6\times$
Descriptor Extraction	2.97 ms	4.60 ms	$1.5\times$
Total	**14.65 ms**	**24.87 ms**	$1.7\times$
Avg. keypoints	*226.9*	*408.5*	$1.8\times$

The results indicate that Non-Maximum Suppression (NMS) is the dominant computational bottleneck, accounting for $56.1\,\%$ of total processing time. Its cost scales with the number of detected keypoints, increasing from 7.58 ms in the detection phase to 14.19 ms in the tracked phase. In contrast, per-pixel operations such as grayscale conversion and frame upload remain relatively constant, highlighting the efficiency of GPU parallelization for dense operations while exposing variability introduced by feature-dependent stages.

7.3 Baseline Systems and Ablation Study

The proposed pipelines are compared against two baselines: a CPU-based algebraic pipeline using homography estimation, and ARCore's native image tracking system.

To isolate the contribution of each component, we perform an ablation study across multiple pipeline variants. The results are summarized in Table 3.

Table 3: Ablation study demonstrating the trade-off between computational frame rate (FPS) and geometric tracking stability. $\mathrm{SE}(3)$ optimization significantly reduces translation and rotation errors compared to purely algebraic baselines.

Pipeline Configuration	**Avg. FPS**	**Trans. Error (px)**	**Rot. Error (deg)**
Baselines			
ARCore Native Image Tracking	30.0	2.14	1.85
CPU Algebraic (Homography only)	58.2	4.30	3.12
Proposed Ablations			
GPU Algebraic (Single-scale)	**60.0**	4.15	3.00
GPU Algebraic (Multiscale)	54.5	3.10	2.45
GPU Geometry-Aware (No LK flow)	42.1	1.45	1.10
GPU Geometry-Aware SE(3) (Full)	50.4	**1.12**	**0.85**

Effect of Geometry-Aware Optimization. The ablation results demonstrate a clear trade-off between computational efficiency and geometric accuracy. Purely algebraic pipelines achieve the highest frame rates but exhibit significantly larger translation and rotation errors. Incorporating 2D–3D correspondence lifting and $\mathrm{SE}(3)$ optimization substantially improves pose stability, reducing both spatial jitter and rotational drift.

Tension-Based Refinement. Within the geometry-aware pipeline, pose refinement is performed using a tension-based weighted Gauss–Newton formulation. Given reprojection residuals:

$$\mathbf{e}_i = \mathbf{x}_i - \pi(\mathbf{K}\mathbf{T}\mathbf{X}_i), \quad d_i = \|\mathbf{e}_i\|,$$

the minimum residual

$$L = \min_i d_i$$

defines a reference equilibrium. Correspondences are assigned adaptive weights:

$$w_i = \frac{d_i - L}{d_i}.$$

The resulting update is computed as:

$$(\mathbf{J}^\top \mathbf{W} \mathbf{J})\, \boldsymbol{\delta} = \mathbf{J}^\top \mathbf{W} \mathbf{e}, \quad \mathbf{T} \leftarrow \exp(\boldsymbol{\delta}^\wedge)\mathbf{T}.$$

This adaptive weighting scheme emphasizes inconsistent correspondences while suppressing already well-aligned matches, improving robustness without requiring manually tuned robust loss functions.

Overall, the results show that while algebraic GPU pipelines maximize raw throughput, the inclusion of geometry-aware refinement—particularly with adaptive tension-based weighting—provides significantly improved tracking stability and accuracy, outperforming both algebraic baselines and standard mobile AR tracking solutions.

8 Discussion

8.1 CPU Pipeline Limitations

The CPU-based pipeline demonstrates strong efficiency at moderate input resolutions but becomes increasingly constrained as image resolution grows. Since feature detection and descriptor extraction scale with the number of pixels, higher resolutions introduce significant computational overhead even with SIMD vectorization.

Reducing camera resolution improves runtime performance but introduces trade-offs in feature quality. At lower resolutions, distant objects occupy fewer pixels, leading to weaker or less distinctive descriptors. This reduces matching reliability, particularly under multiscale conditions, and can negatively impact tracking robustness.

8.2 GPU Pipeline Trade-offs

The GPU-accelerated pipeline alleviates many of the computational bottlenecks associated with feature extraction and descriptor computation by exploiting data-parallel execution. However, its performance remains dependent on memory transfer costs and overall GPU availability.

Although the use of storage buffers significantly reduces data transfer overhead compared to transferring full image buffers, the cost of transferring match data still scales with the number of correspondences. As a result, scene complexity directly influences runtime performance.

Additionally, GPU performance on mobile platforms is subject to system-level constraints. Since the GPU is often shared with rendering and other application processes, scheduling variability and resource contention can reduce the effective throughput available for feature processing.

8.3 Geometric Tracking Advantages

The proposed $\mathrm{SE}(3)$ render-and-track formulation addresses several limitations of purely algebraic pipelines by incorporating explicit geometric constraints derived from the object model. The availability of object-space coordinates during rendering enables direct lifting of image correspondences to 2D–3D constraints, reducing sensitivity to clustered or spatially limited feature distributions.

Furthermore, the tension-based refinement introduces adaptive correspondence weighting within the optimization process. By assigning weights based on relative residual magnitudes, the system emphasizes inconsistent correspondences while suppressing those already in agreement. This results in improved robustness under partial visibility, noise, and viewpoint changes, without requiring manually tuned robust loss functions.

8.4 Comparison with ARCore

ARCore provides a highly optimized mobile tracking framework with built-in image tracking capabilities [22]. While it achieves strong performance in marker-based scenarios, it operates as a closed system with limited control over the

underlying tracking pipeline. Consequently, its performance may degrade under strong viewpoint changes or partial occlusions, depending on the visual properties of the tracked image.

In contrast, the proposed system offers explicit control over all stages of the tracking pipeline and enables the integration of geometry-aware constraints through render-and-track optimization. This flexibility allows the system to maintain stable pose estimates in scenarios where purely appearance-based tracking methods struggle.

8.5 Future Work

While the proposed framework focuses on feature-based tracking using keypoint correspondences, several extensions remain possible. One promising direction is the integration of silhouette-based tracking methods. Keypoint-based approaches rely on the presence of distinctive local features, which may become sparse under low-texture conditions, motion blur, or significant scale variation.

Silhouette-based tracking, which aligns projected object contours with observed image edges, can provide complementary geometric constraints that are less dependent on texture. Incorporating silhouette alignment within the render-and-track framework could improve robustness in challenging scenarios, particularly under partial visibility or low-texture surfaces.

Another direction for future work is the exploration of hybrid optimization strategies that combine feature-based residuals with additional geometric cues. Extending the tension-based formulation to incorporate multiple constraint types may further improve robustness while maintaining computational efficiency.

Investigating these extensions within the proposed framework represents a promising direction for improving real-time tracking performance on resource-constrained devices.

9 Conclusion

This paper presented a hardware-aware framework for real-time 6-DoF object tracking on mobile and embedded devices. The system explores three progressively more capable tracking configurations: a CPU-based algebraic pipeline, a GPU-accelerated feature processing pipeline, and a geometry-aware render-and-track formulation operating on the $\mathrm{SE}(3)$ pose manifold.

Experimental evaluation on mobile devices highlights the trade-offs between these configurations. While the CPU-based pipeline provides an efficient baseline, its performance degrades with increasing image resolution. GPU acceleration significantly improves feature processing throughput, but introduces constraints related to memory transfer overhead and GPU resource utilization.

To address the limitations of purely algebraic approaches, the proposed geometry-aware pipeline incorporates rendering-based 2D–3D correspondence lifting and pose refinement on $\mathrm{SE}(3)$. In particular, the tension-based formulation introduces adaptive correspondence weighting within a weighted Gauss–Newton framework, improving robustness to noise, outliers, and partial visibility without requiring manually tuned loss functions.

This combination of hardware-aware computation and geometry-aware optimization results in improved pose stability under challenging conditions such as significant viewpoint variation and limited feature visibility. Although the geometry-aware stage introduces additional per-frame computation, it reduces drift and the need for frequent relocalization, leading to more consistent tracking performance over time.

Overall, the results demonstrate that integrating adaptive, geometry-driven optimization with efficient hardware utilization enables robust and scalable real-time tracking on resource-constrained edge devices. This approach provides a practical alternative to both purely algebraic pipelines and computationally intensive SLAM or learning-based methods for mobile augmented reality applications.

References

[1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "Orb-slam: A versatile and accurate monocular slam system," *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015.

[2] J. Engel, T. Schöps, and D. Cremers, "Lsd-slam: Large-scale direct monocular slam," in *Proc. European Conference on Computer Vision (ECCV)*, pp. 834–849, 2014.

[3] R. A. Newcombe, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A. J. Davison, P. Kohli, J. Shotton, S. Hodges, and A. Fitzgibbon, "Kinectfusion: Real-time dense surface mapping and tracking," in *Proc. IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pp. 127–136, 2011.

[4] G. Klein and D. Murray, "Parallel tracking and mapping for small ar workspaces," in *Proc. IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pp. 225–234, 2007.

[5] V. Miu, "Real-time monocular 6dof object pose tracking on smartphone gpus," in *Proc. European Conference on Visual Media Production (CVMP)*, 2020.

[6] J. Rambach, A. Pagani, M. Schneider, O. Artemenko, and D. Stricker, "6dof object tracking based on 3d scans for augmented reality remote live support," in *Computers*, vol. 7, p. 6, 2018.

[7] J. Solà, J. Deray, and D. Atchuthan, "A micro lie theory for state estimation in robotics," 2018.

[8] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.

[9] C. Choi and H. I. Christensen, "3d textureless object detection and tracking: An edge-based approach," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3872–3879, 2012.

[10] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "Orb: An efficient alternative to sift or surf," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, pp. 2564–2571, 2011.

[11] S. Leutenegger, M. Chli, and R. Siegwart, "Brisk: Binary robust invariant scalable keypoints," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, pp. 2548–2555, 2011.

[12] E. Rosten and T. Drummond, "Machine learning for high-speed corner detection," in *Proc. European Conference on Computer Vision (ECCV)*, pp. 430–443, 2006.

[13] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[14] L. Kneip, D. Scaramuzza, and R. Siegwart, "A novel parametrization of the perspective-three-point problem," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2609–2616, 2011.

[15] X.-S. Gao, X.-R. Hou, J. Tang, and H.-F. Cheng, "Complete solution classification for the perspective-three-point problem," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 25, no. 8, pp. 930–943, 2003.

[16] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2 ed., 2003.

[17] B. Triggs, P. F. McLauchlan, R. I. Hartley, and A. W. Fitzgibbon, "Bundle adjustment — a modern synthesis," in *Proc. International Workshop on Vision Algorithms*, pp. 298–372, 1999.

[18] J. Nocedal and S. J. Wright, *Numerical Optimization*. Springer, 2 ed., 2006.

[19] P. J. Huber, "Robust estimation of a location parameter," *Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[20] M. J. Black and A. Rangarajan, "On the unification of line processes, outlier rejection, and robust statistics," *International Journal of Computer Vision*, vol. 19, no. 1, pp. 57–92, 1996.

[21] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proc. International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 674–679, 1981.

[22] Google LLC, "Arcore overview." https://developers.google.com/ar, 2026.